

系统开发工具基础实验报告（第四周）

廖世嘉 25020007073
25 级计算机科学与技术

2026 年 9 月 14 日

目录

1	实验基本信息	3
2	实验目的	3
3	实验环境与工具	3
4	课上检查内容与结果	3
4.1	实验十三：建立可执行的本地质量门禁	3
4.1.1	实验要求	3
4.1.2	配置文件	4
4.1.3	测试用例	4
4.1.4	质量检查脚本	4
4.1.5	结果	5
4.2	实验十四：让 Make 只重建真正受影响的产物	5
4.2.1	实验要求	5
4.2.2	源文件	5
4.2.3	Makefile	6
4.2.4	验证结果	6
4.3	实验十五：把本地 API 数据转换为可读报告	6
4.3.1	实验要求	6
4.3.2	数据文件	7
4.3.3	脚本实现	7
4.3.4	结果	7
4.4	实验十六：修复并交付一个陌生的小型工具仓库	8
4.4.1	实验要求	8
4.4.2	临时破坏与验证	8
4.4.3	修复	8
4.4.4	Makefile	8
4.4.5	构建与交付	9

目录	2
5 课后练习内容与结果	9
5.1 讲义一：代码质量 (code-quality)	9
5.1.1 练习 1 配置格式器、linter 与 pre-commit 门禁	9
5.1.2 练习 2 单元测试与 HTML 覆盖率	9
5.1.3 练习 3 CI 与”故意弄坏”验证	10
5.1.4 练习 4 用正则找危险 shell 调用，semgrep 兜底	10
5.1.5 练习 5 正则查找替换项目符号	10
5.1.6 练习 6 从 JSON 捕获 name 与命令行解析器	11
5.2 讲义二：元编程 (metaprogramming)	11
5.2.1 练习 7 Makefile: git ls-files 驱动与.PHONY	11
5.2.2 练习 8 Rust 版本语义	11
5.2.3 练习 9 pre-commit 钩子：构建失败拒绝提交	12
5.2.4 练习 10 GitHub Pages 与 shellcheck Action	12
5.2.5 练习 11 自定义 GitHub Action 对.md 跑 proselint	12
5.3 课后练习汇总	13
6 解题感悟	13
7 问题与解决方法	14
7.1 课上部分	14
7.1.1 ruff 导入排序问题	14
7.1.2 Makefile 中的 Tab 缩进	14
7.1.3 jq 排序中的多字段排序	14
7.1.4 Git 提交的聚焦性	14
7.2 课后部分	14
7.2.1 网络代理与依赖拉取	14
7.2.2 gcov/lcov 版本兼容	14
7.2.3 cppcheck 对 gtest 宏的解析缺陷	15
7.2.4 git restore 的暂存区陷阱	15
7.2.5 proselint 空参数读 stdin	15
8 实验总结	15
9 GitHub 链接与 commit 记录	15

1 实验基本信息

表 1: 实验基本信息

项目	内容
姓名	廖世嘉
学号	25020007073
专业	25 级计算机科学与技术
实验环境	WSL Ubuntu 26.04: 课上部分使用 Python/ruff/pytest/Make/curl/jq/LaTeX; 课后部分在 C++20/CMake 项目 MudGame 上完成, 使用 clang-format、cppcheck、 pre-commit、lcov、semgrep、actionlint、shellcheck、cargo、latexmk
实验日期	2026 年 9 月 14 日
课程仓库	https://github.com/MrLLLiao/SystemToolkitDevCourse
练习仓库	https://github.com/MrLLLiao/MudGame (课后练习所在, 仅本地提交, 未推送)

2 实验目的

第四周课上讲代码质量与元编程。课上检查的四道题 (q13–q16) 按讲义要求, 在 greetlab 包上搭本地质量门禁、写 Makefile、用 curl+jq 出报告、最后做一次综合交付。课后练习把同一批工具搬到一个更大的 C++ 项目里跑一遍: 给 MudGame 配上格式检查、静态分析、覆盖率、CI、semgrep、自定义 GitHub Action, 再补一个 Makefile 和 Rust 版本语义的演示。目的很直接: 课上学过的工具在真实仓库里是否立得住, 哪些地方会卡住, 用了才知道。

3 实验环境与工具

课上部分沿用前几周的 Python 环境: ruff 0.x、pytest、Make、curl、jq、latexmk。q13–q16 的代码在课程仓库 SystemToolkitDevCourse/week4 下的 q13–q16 目录中。

课后 11 个练习全部在 MudGame 仓库 (C++20 / CMake, 21 个单元测试) 中完成。WSL 侧工具链版本: clang-format 21.1.8、cppcheck 2.19.0、lcov 2.0-1、pre-commit 4.6.2、semgrep 1.176.1、actionlint 1.7.12、shellcheck 0.11.0、cargo/rustc 1.93.1、jq 1.8.1。练习期间发现宿主 git 全局代理被设置成 WSL 内不可达的 127.0.0.1:7890, 凡涉及 FetchContent 拉取依赖的命令都临时用 GIT_CONFIG_COUNT 覆盖为宿主代理, 未改动全局配置。

4 课上检查内容与结果

4.1 实验十三: 建立可执行的本地质量门禁

4.1.1 实验要求

复制 q10 为 q13, 在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。补充一个正常姓名测试和一个空白姓名测试, 每个测试包含有效断言。运行 ruff format、ruff check 和 pytest, 修复全部问题, 不得全局忽略规则。编写 check.sh, 依次执行 ruff format –check、ruff check 和 pytest, 运行并保证返回 0。

4.1.2 配置文件

pyproject.toml 中新增 ruff 和 pytest 配置:

```
[tool.ruff]
line-length = 88
target-version = "py310"

[tool.ruff.lint]
select = ["E", "F", "W", "I"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["."]
```

4.1.3 测试用例

test_cli.py 包含两个测试:

```
import pytest
from greetlab.cli import main

def test_normal_name(capsys):
    import sys
    sys.argv = ["sdt-greet", "--name", "alice"]
    main()
    out = capsys.readouterr().out
    assert "Hello, alice!" in out

def test_blank_name_exits_nonzero():
    import sys
    sys.argv = ["sdt-greet", "--name", " "]
    with pytest.raises(SystemExit) as exc_info:
        main()
    assert exc_info.value.code == 2
```

4.1.4 质量检查脚本

check.sh 依次执行三项检查:

```
#!/usr/bin/env bash
set -euo pipefail

echo "=== ruff format --check ==="
ruff format --check .

echo "=== ruff check ==="
ruff check .

echo "=== pytest ==="
python -m pytest test_cli.py -v
```

```
echo "=== All checks passed ==="
```

4.1.5 结果

运行 check.sh 输出：

```
=== ruff format --check ===
4 files already formatted
=== ruff check ===
All checks passed!
=== pytest ===
2 passed
=== All checks passed ===
```

退出码为 0，全部检查通过。期间 ruff 发现的导入排序问题已通过 `ruff check -fix` 修复，未使用全局忽略规则。

4.2 实验十四：让 Make 只重建真正受影响的产物

4.2.1 实验要求

在 q14 中创建 data.csv、stats.py、build_report.py 和 report.md 四个源文件，编写 Makefile 包含 all、stats.txt、report.txt 和 clean 目标。准确列出数据文件、脚本和中间产物依赖，将 all 和 clean 声明为.PHONY。验证首次 make 生成两个产物，第二次无改动时不执行配方，touch data.csv 后重新生成，clean 只删除生成物。

4.2.2 源文件

```
# data.csv
name,value
alpha,2
beta,3
gamma,5
```

stats.py 读取 data.csv 并计算总和写入 stats.txt：

```
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))
```

build_report.py 将 report.md 和 stats.txt 合并为 report.txt：

```
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
```

4.2.3 Makefile

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: report.md stats.txt build_report.py
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
```

依赖链: data.csv 和 stats.py 生成 stats.txt, report.md、stats.txt 和 build_report.py 生成 report.txt。all 依赖 report.txt, clean 删除两个生成物。

4.2.4 验证结果

```
# 首次 make
$ make
python3 stats.py
python3 build_report.py

# 第二次 make (无改动)
$ make
make: Nothing to be done for 'all'.

# touch data.csv 后
$ touch data.csv && make
python3 stats.py
python3 build_report.py

# clean
$ make clean
rm -f stats.txt report.txt
```

首次 make 生成两个产物, 第二次 make 因无改动而不执行配方, touch data.csv 后 stats.txt 和 report.txt 均重新生成, clean 只删除生成物, 保留源文件。Make 的增量构建机制工作正确。

4.3 实验十五: 把本地 API 数据转换为可读报告

4.3.1 实验要求

在 q15 中创建给定的 packages.json, 运行 `python -m http.server 8000` 启动本地 HTTP 服务。使用 `curl -fsS` 获取数据, 使用 `jq` 筛选 status 为 active 且 downloads 不少于 100 的记录, 按 downloads 降序、name 升序排列。编写 `api_report.sh` 将结果生成 `summary.md`, 包含标题和 name/version/downloads 三列表格。

4.3.2 数据文件

packages.json 包含 5 条记录:

```
[
  {"name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0"},
  {"name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0"},
  {"name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1"},
  {"name": "delta", "status": "active", "downloads": 450, "version": "0.9.0"},
  {"name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"}
]
```

4.3.3 脚本实现

api_report.sh 使用 curl 获取 JSON 数据, 通过 jq 进行筛选、排序和 Markdown 表格生成:

```
#!/usr/bin/env bash
set -euo pipefail

URL="http://127.0.0.1:8000/packages.json"
OUTPUT="summary.md"

curl -fsS "$URL" | \
  jq -r '
    ["| name | version | downloads |", "|-----|-----|-----|"] as $header |
    [$header[],
      ([.[] | select(.status == "active" and .downloads >= 100)]
       | sort_by(-.downloads, .name)
       | .[]
       | "| \(.name) | \(.version) | \(.downloads) |")
    ] | .[]' > "$OUTPUT"

{
  echo "# Package Summary"
  echo ""
  cat "$OUTPUT"
} > "${OUTPUT}.tmp" && mv "${OUTPUT}.tmp" "$OUTPUT"

echo "Report written to $OUTPUT"
```

4.3.4 结果

在 q15 目录启动 HTTP 服务后运行 api_report.sh, 生成的 summary.md:

```
# Package Summary

| name | version | downloads |
|-----|-----|-----|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

筛选结果正确: beta (inactive) 和 epsilon (downloads=80 < 100) 被排除。delta 和 gamma 的 downloads 均为 450, 按 name 升序 delta 排在 gamma 之前。curl -fsS 在服务不可用时返回非零退出码, 确保错误可被脚本捕获。

4.4 实验十六: 修复并交付一个陌生的小型工具仓库

4.4.1 实验要求

复制 q13 为 q16 并初始化 Git。把问候语实现临时改成始终返回字面量 Hello, name!, 运行 check.sh 确认测试失败, 随后定位并修复。编写最小 Makefile 包含 check、build 和 clean, check 调用 check.sh, build 生成 wheel。运行 make check 和 make build, 计算 wheel 的 SHA-256, 并把最终改动提交为一个内容聚焦的 Git 提交。

4.4.2 临时破坏与验证

将 cli.py 中的实现临时改为字面量输出:

```
# 临时改坏
print("Hello, name!")
```

运行 check.sh 确认测试失败:

```
=== ruff check ===
F841 Local variable `a` is assigned to but never used
=== pytest ===
FAILED test_normal_name - assert "Hello, alice!" in "Hello, name!"
FAILED test_blank_name_exits_nonzero - Failed: DID NOT RAISE SystemExit
```

4.4.3 修复

将实现恢复为正确版本:

```
if not a.name.strip():
    raise SystemExit(2)
print(f"Hello, {a.name}!")
```

4.4.4 Makefile

```
.PHONY: check build clean

check:
    bash check.sh

build:
    python3 -m build --no-isolation

clean:
    rm -rf build dist src/*.egg-info .pytest_cache .ruff_cache
```


4.4.5 构建与交付

运行 `make check` 全部通过，运行 `make build` 成功生成 wheel。计算 SHA-256：

```
$ sha256sum dist/*.whl
f5acb7abe8a6604d88a68cedc447857cf9b7add89e04c4a154ca489b630d1c6 dist/greetlab_25020007073
-0.1.0-py3-none-any.whl
```

Git 提交记录：

```
$ git log --oneline
778d461 fix: restore correct greeting and add Makefile with check/build/clean
c4cd828 init: greetlab package from q13
```

提交信息包含问题说明、修复内容和 SHA-256 哈希值，内容聚焦于本次修复和构建配置。

5 课后练习内容与结果

对应讲义 [code-quality \(2026\)](#) 与 [metaprogramming \(2020\)](#) 两页，共完成 11 个实例。全部在 MudGame 仓库 (C++20/CMake) 上完成，按练习分步提交，共 18 个 commit，commit 信息格式为 `feat(xxx)`：中文内容，未推送远端。

5.1 讲义一：代码质量 (code-quality)

5.1.1 练习 1 配置格式器、linter 与 pre-commit 门禁

内容：给 C++ 项目配置 clang-format、cppcheck，并接上 pre-commit 钩子，让不合格的代码提交不进去。

做法：新增 `.clang-format` (BasedOnStyle LLVM，缩进 4 空格，Allman 大括号，对齐仓库现有风格)、`.cppcheck-suppressions` (unusedFunction、missingIncludeSystem 等文件级抑制) 和 `.pre-commit-config.yaml` (三个 local 钩子：clang-format 检查暂存 C/C++ 文件、cppcheck 检查暂存 src/ 文件、增量构建 +ctest)。

结果：全量扫描时发现 1628 处存量格式违规，直接全库格式化会一次产生超大 diff，也不符合“分步提交”的要求，所以采用增量采用策略——门禁只检查 staged 文件，存量问题留给后续逐步清理。钩子第一次真正拦截发生在 `time_service.cpp`：cppcheck 报了 performance 建议 (passed-ByValue)，加抑制并说明理由后通过。练习期间还发现 clang-format 不能用于 CMake 文件 (会把 `add_executable` 格式坏)，已用 git checkout 恢复。

提交：d789ea9 (门禁与钩子)、d8b79c3 (time_service 格式化 + 抑制)。

5.1.2 练习 2 单元测试与 HTML 覆盖率

内容：用 lcov + genhtml 生成覆盖率 HTML 报告，找出未覆盖代码并补测试。

做法：独立构建目录 build-cov 加 `-DCMAKE_CXX_FLAGS='--coverage -O0 -g'`，跑完 ctest 后 lcov capture、按 `*/MudGame/src/*` extract，再 genhtml 输出到 coverage/。gcc 15 的 gcov 数据与 lcov 2.0 存在兼容告警，命令必须带 `--rc branch_coverage=1 --ignore-errors mismatch,inconsistent,unused`，否则 capture 直接报错。脚本固化在 `scripts/coverage.sh`。

结果：初值行覆盖 86.4% (386/447)、函数 81.1% (73/90)、分支 52.4% (297/567)。对照未覆盖行清单，补了两个测试文件：ore_lookup_test.cpp 覆盖 Ore/Layer 查询的异常路径 (未知矿石/未知矿

区抛 `out_of_range`、产出权重回退 0.0) 和 `requires_lighting` 的真假分支; `tool_upgrade_test.cpp` 覆盖 `Tool::upgrade` 的满级与升级分支。补充后行覆盖 93.5% (418/447)、函数 91.1% (82/90)、分支 60.7% (344/567)。其中 `object.h` 的未初始化成员告警被顺带修复 (见练习 3)。

提交: ff18905。

5.1.3 练习 3 CI 与”故意弄坏”验证

内容: 配置 GitHub Actions 工作流, 并故意弄坏代码验证 CI 能抓到。

做法: `.github/workflows/ci.yml` 两个 job: `build-test` (安装依赖、cmake 配置、构建、ctest) 和 `quality` (对**变更文件**跑 `clang-format -dry-run -Werror` 与 `cppcheck`, 增量策略与本地门禁一致)。用 `actionlint` 本地校验通过。

结果: 本地模拟 CI 的三次破坏: 1 把 `mining_layers.json` 中 `shallow` 层解锁等级从 1 改成 99 → ctest 失败 (`ContextGate.StartIdleEntersMining` 等用例被数据破坏触发); 2 给 `object.h` 插入错误缩进 → `clang-format -dry-run -Werror` 返回 1; 3 还原后 21/21 全绿。另外 `cppcheck` 全量扫描 `src/` 时发现 `object.h` 的 `sellingPrice/buyingPrice` 未初始化 (默认构造后取值是未定义行为), 修复并格式化后单独提交。

提交: 1581024 (`object.h` 修复)、6a9e4b3 (CI 工作流)。

5.1.4 练习 4 用正则找危险 shell 调用, semgrep 兜底

内容: 按讲义在代码库里搜 `subprocess.Popen(..., shell=True)` 一类的危险调用, 先体验正则的真实代码前的失效方式, 再用 `semgrep` 兜底。

做法与结果 (在 `/tmp` 样例上演示, 生产代码本身无 `system/popen` 调用):

```
# 正则1 (讲义式): 只命中单行调用, 多行 Popen 被漏掉
$ grep -nE 'subprocess\.Popen\([^\)]*shell\s*=\s*True' demo.py
5: subprocess.Popen("ls -la", shell=True)

# 正则2: os.system 后接行续接符再换行, \bsystem\s*\(\s* 漏报
$ grep -nE '\bsystem\s*\(\s*' demo3.py
(0 命中)

# 破坏正则 (去掉 =\s*True): shell=False 的安全调用被误报
$ grep -nE 'subprocess\.Popen\([^\)]*shell' demo.py
16: subprocess.Popen(["ls", "-la"], shell=False) # 误报

# semgrep 自定义规则: AST 解析, 两种危险形态都命中, 无误报
$ semgrep --config danger.yml demo.py
Findings: 3 (Popen 单行、Popen 多行、os.system 多行)
```

结论: 正则写复杂了难维护、写简单了漏报误报; `semgrep` 按语法树匹配, 同样的规则语义下结果稳定。对安全相关检查, 工具选型优先级高于正则技巧。

提交: 无 (技术演示)。

5.1.5 练习 5 正则查找替换项目符号

内容: 把 Markdown 中 “-” 项目符号统一替换为 “*”。

表 2: Rust 版本语义实测 (rand)

版本要求	解析结果	说明
0.8	0.8.8	caret: $\geq 0.8.0$ 且 $< 0.9.0$ (0.x 锁次版本)
0.8.3	0.8.8	caret: $\geq 0.8.3$ 且 $< 0.9.0$
~0.8.4	0.8.8	tilde: $\geq 0.8.4$ 且 $< 0.9.0$
0.8.*	0.8.8	通配: 仅 0.8.x
$\geq 0.8, < 0.9$	0.8.8	范围: 多约束交集
=0.8.4	0.8.4	精确匹配
$\geq 1.0, < 1.5$	解析失败	"no matching version"
$\geq 0.8.5$	0.10.2	无上限: 直接选最新

构建的骰子场景 (rand 0.8 的 gen_range API) 编译通过, 连续运行输出点数 6、2、6、5; cargo tree 显示 rand 0.8.8 及其传递依赖。

提交: 34f8759 (演示项目)、968fb03 (提交 Cargo.lock 保证可复现构建)。

5.2.3 练习 9 pre-commit 钩子: 构建失败拒绝提交

内容: 让 pre-commit 钩子在构建失败时拒绝提交 (对应讲义 "make paper.pdf 失败则不提交")。

做法: 练习 1 已有 build+ctest 钩子, 这次把钩子内部改为走练习 7 的 Makefile (make build && make test), 让两处复用同一套构建入口。

结果: 故意往 object.cpp 追加一行语法错误后提交, pre-commit 依次拦截: cppcheck 报 syntax-Error (exit 2), build 钩子编译失败 (exit 1), 提交被拒绝。还原文件后提交正常通过。演示中踩了一个坑: git restore 只还原工作树不还原暂存区, 暂存区仍是坏版本, 需要 git restore --source=HEAD --staged --worktree 才能彻底还原。

提交: da08f6f。

5.2.4 练习 10 GitHub Pages 与 shellcheck Action

内容: 给仓库配置 GitHub Pages 部署工作流, 并在 CI 中加入 shellcheck 步骤检查 shell 脚本。

做法与结果: shellcheck 0.11.0 本地检查 scripts/ 下 4 个脚本全部通过。ci.yml 的 quality job 增加 shellcheck 步骤 (apt 安装后 shellcheck -x scripts/*.sh)。新建 .github/workflows/pages.yml (configure-pages + upload-pages-artifact + deploy-pages) 与 docs/index.html 静态站点, 推送 master 后自动发布到 Pages (需在仓库 Settings 中把发布源设为 GitHub Actions)。两份工作流都通过 actionlint 校验。

提交: 8503c01 (shellcheck)、74a7e54 (Pages)。

5.2.5 练习 11 自定义 GitHub Action 对.md 跑 proselint

内容: 写一个自定义复合 Action, 对仓库 markdown 文件运行 proselint 并报告文体违规, 在.md 变更时触发。

做法: .github/actions/proselint-check/action.yml (composite action: 安装 proselint 后对传入文件运行); md-lint.yml 计算本次变更的.md 文件 (与 CI 其他检查相同的增量策略) 再调用该 Action。

结果：proselint 对 README.md 报告 4 处 “...” 应为 “...” 的标点问题，修复后本地等价命令通过。期间发现 proselint check 无参数时会读取 stdin 卡死，Action 内加了空文件守卫（无变更则跳过）。

提交：ae2682e（标点修复）、d55253d（Action 与 workflows）、10e3774（改为只检查变更文件）。

5.3 课后练习汇总

表 3: 11 个课后练习完成情况

编号	讲义	练习主题	验证结果
1	code-quality	格式器 +linter+pre-commit 门禁	增量门禁生效，两次真实拦截
2	code-quality	单元测试与 HTML 覆盖率	行覆盖 86.4%→93.5%，报告已生成
3	code-quality	CI 与故意破坏验证	数据/格式破坏均被 CI 抓到
4	code-quality	regex 找危险调用 + semgrep 兜底	正则漏报/误报，semgrep 全抓无误报
5	code-quality	正则替换项目符号	4 行 “-”→“*”，锚定无误伤
6	code-quality	JSON 捕获 name + 命令行解析器	贪婪/懒惰/转义三例对比，jq 与脚本正确
7	metaprogramming	Makefile (git ls-files + .PHONY)	30 源文件，build/test/clean 验证通过
8	metaprogramming	Rust 版本语义	8 种约束实测，不可满足正确报错
9	metaprogramming	pre-commit 拒绝构建失败	语法错误提交被双钩子拦截
10	metaprogramming	GitHub Pages + shellcheck	4 脚本全过，工作流 actionlint 通过
11	metaprogramming	自定义 Action 跑 proselint	报告 4 处标点问题，修复后通过

6 解题感悟

这周是四周里动手最多的一次。课上 q13-q16 的每个工具都小而确定，按讲义一步步走就能过；课后把它们搬到 MudGame 上，才体会到工具链真正的工作量分布：写检查规则本身不难，难在让检查跑通、并且不误伤。

第一个感受是**门禁要“增量”而不是“全量”**。MudGame 存量代码有 1628 处格式违规，如果按讲义默认做法让 clang-format 检查全库，第一次提交就会被淹没。改成只检查 staged 文件后，每个新提交都是干净的，存量债可以慢慢还。这个取舍在 CI 里同样成立：quality job 只查变更文件，否则 CI 从第一天起就是红的，也就失去意义了。

第二个感受是**工具链的坑比工具本身更花时间**。几件小事：宿主 git 全局代理指向 WSL 内不可达的 127.0.0.1，FetchContent 拉 CLI11 必失败，最后用 GIT_CONFIG_COUNT 临时覆盖；gcc 15 的 gcov 和 lcov 2.0 数据格式不兼容，不加 ignore-errors 覆盖率报告根本出不来；cargo 不认 HTTP_PROXY 只认 CARGO_HTTP_PROXY；proselint 无参数会读 stdin 卡死。每个坑都不深，但排查路径各不相同——读报错、试参数、看文档，一步步来。

第三个感受是**覆盖率数字要落到“没测到哪”才有意义**。初值行覆盖 86.4% 看着不低，但分支只有 52.4%。对照未覆盖行清单发现漏的全是错误路径：未知矿石要抛异常、未知矿区权重要回退 0、工具满级后 upgrade 要返回 false。补上这些用例后分支到 60.7%，而这些恰好是游戏运行中最容易崩的地方。

第四点是**分步提交的价值**。18 个 commit 每个只做一件事，门禁日志、覆盖率脚本、CI、semgrep 演示各自独立。中间两次提交被钩子拦下、一次被 cppcheck 语法误报拦下，回看 git log 时每一条

都能说清意图，这在一次性全量提交里是做不到的。

最后，练习 4 和练习 6 让我对正则的态度变了。以前写正则喜欢“一步到位”，现在知道安全相关的匹配要交给按语法树解析的工具（`semgrep`），数据提取要交给真正的解析器（`jq` / `json` 模块），正则适合做定位和快速过滤，不适合做需要精确语义的检查。

7 问题与解决方法

7.1 课上部分

7.1.1 ruff 导入排序问题

在实验十三中，初次运行 `ruff check` 发现 `test_cli.py` 的导入顺序不符合 `isort` 规则（I001）。使用 `ruff check -fix` 自动修复后，再次检查全部通过，无需全局忽略规则。

7.1.2 Makefile 中的 Tab 缩进

在实验十四中，`Makefile` 的配方行必须使用 `Tab` 而非空格缩进。在 Windows 编辑器中保存时，有时会将 `Tab` 转换为空格，导致 `make` 报错“missing separator”。解决方法是使用 `cat -A` 检查 `Makefile` 中的缩进字符，确保配方行以 `<TAB>` 开头。

7.1.3 jq 排序中的多字段排序

在实验十五中，需要先按 `downloads` 降序、再按 `name` 升序排列。`jq` 的 `sort_by` 函数默认按升序排列，对于降序需要在字段前加负号：`sort_by(-.downloads, .name)`。这一语法确保 `delta` 和 `gamma` 在相同 `downloads`（450）时按 `name` 升序排列。

7.1.4 Git 提交的聚焦性

在实验十六中，初始提交误将临时文件 `commit_msg.txt` 一并提交。通过 `git rm --cached` 移除该文件并 `amend` 提交，确保最终提交仅包含 `.gitignore` 和 `Makefile` 两个必要文件的变更。

7.2 课后部分

7.2.1 网络代理与依赖拉取

WSL 内 `git` 全局代理指向 `127.0.0.1:7890` (WSL 内不可达), `CMake FetchContent` 拉取 `CLI11/pjh_json/google` 全部失败。解决:命令前注入 `GIT_CONFIG_COUNT=1 GIT_CONFIG_KEY_0=http.proxy GIT_CONFIG_VALUE_0=http://` 覆盖代理 (不改全局配置); `cargo` 则需设置 `CARGO_HTTP_PROXY`。

7.2.2 gcov/lcov 版本兼容

`gcc 15` 的 `gcov` 数据与 `lcov 2.0` 报 `mismatch/inconsistent` 告警, 不带参数时 `lcov capture` 直接失败。解决: `capture/extract` 均加 `--rc branch_coverage=1 --ignore-errors mismatch,inconsistent,unused`, 固化进 `scripts/coverage.sh`。

7.2.3 cppcheck 对 gtest 宏的解析缺陷

cppcheck 2.19 对 gtest 的 TEST 宏内 while(成员调用)、EXPECT_THROW 等写法报 syntaxError (代码本身编译和运行都正常, 已用最小样例复现)。语法错误无法用 suppression 屏蔽, 最终把门禁的 cppcheck 限定为只分析生产代码 src/, 测试代码由编译器与测试运行保障。

7.2.4 git restore 的暂存区陷阱

练习 9 演示还原时, git restore 默认只还原工作树, 暂存区仍保留坏版本, 导致提交继续失败。需要 `git restore --source=HEAD --staged --worktree` 同时还原两侧。

7.2.5 proselint 空参数读 stdin

proselint check 不带文件参数时会从 stdin 读取并阻塞。自定义 Action 与本地验证脚本都必须先判断文件列表为空则跳过。

8 实验总结

课上四道题把质量门禁、Make 增量构建、curl+jq 数据处理、综合交付完整走了一遍, 流程上没有卡点。课后 11 个练习把它们放到真实的 C++ 仓库里, 得到三个结论: 第一, 质量门禁必须增量生效, 否则大仓库第一天就是红的; 第二, 覆盖率报告的价值在未覆盖清单, 而不在百分比本身; 第三, 正则、静态分析器、构建工具、CI 各有适用边界, 选对工具比把单个工具用得更花哨重要。四周的工具在这周汇到了一起: git 分步提交、pre-commit 门禁、CI 自动化、覆盖率与静态分析, 从“代码能跑”走到了“改动可验证”。

9 GitHub 链接与 commit 记录

课程仓库: <https://github.com/MrLLLiao/SystemToolkitDevCourse> (本报告与 q13-q16 源码所在)。

练习仓库: <https://github.com/MrLLLiao/MudGame> (11 个课后练习所在, 全部改动按练习分步提交, 未做单次全量提交)。

MudGame 本次练习的 commit 记录 (最新在上):

```

* 10e3774 feat(ci): 自定义proselint Action仅检查变更md文件 (增量策略)
* d55253d feat(ci): 自定义proselint复合Action与md-lint workflow
* ae2682e fix(doc): 修正README省略号标点 (proselint报告)
* 74a7e54 feat(ci): 新增GitHub Pages部署 workflow与静态站点
* 8503c01 feat(ci): CI增加shellcheck脚本检查步骤
* da08f6f feat(toolchain): 构建门禁改用Makefile封装
* 968fb03 feat(rust): 提交Cargo.lock保证依赖可复现构建
* 34f8759 feat(rust): 新增Rust版本语义演示项目, 对比caret/tilde/通配/范围/精确约束
* 715553e feat(build): 新增Makefile封装, git ls-files驱动清单与PHONY伪目标
* bb2fb43 feat(tool): 新增命令行JSON解析小工具, 替代脆弱的正则提取
* 53b830a feat(doc): 正则批量将项目符号-替换为*
* 1cc8bd9 feat(doc): README补充功能特性清单 (使用-项目符号)
* 6a9e4b3 feat(ci): 新增GitHub Actions workflow, 构建测试与变更文件格式/lint门禁
* 1581024 fix(object): 初始化sellingPrice/buyingPrice成员并统一格式, 修复cppcheck未初始化告警
* ff18905 feat(test): 补充矿石/矿区查询边界与工具升级测试, 覆盖率提升至93.5%; cppcheck限定生产代码
* d8b79c3 feat(style): 应用clang-format统一时间服务模块风格, 门禁逐行抑制cppcheck误报
* d789ea9 feat(toolchain): 新增clang-format与cppcheck质量门禁及pre-commit钩子
* 3c48cb4 fix(build): 修正tool_controller大小写引用错误, 修复构建失败
* 91dd29e Merge pull request #2 from CarrotMissileVehicle/feature-time
| \
| * 4950aff chore(docs): 根目录文档移入docs, 删除tests过期的case文档与README

```

图 1: MudGame 课后练习 commit 记录 (git log --graph --oneline)